GUÍA DE PLANIFICACIÓN

Sitio web de la serie: página principal y página de personajes

Desarrollo de Aplicaciones Web · Práctica 3 (G003-S10) · Grupo 21

Dato	Detalle
Tipo de trabajo	Práctica grupal, 6 integrantes, evaluada sobre 15 puntos
Entregables	Informe en PDF subido a la plataforma y proyecto del sitio web anexado (se verifica desde el computador del docente)
Tecnologías	HTML, CSS (con Bootstrap) y JavaScript. Sin base de datos y sin frameworks tipo Angular, React o Vue
Fuente de datos	API pública Game of Thrones API: https://thronesapi.com/
Repositorio	https://github.com/MateoACantu/DAW_GRUPO_21_S10-TPE_3.git
Estado del documento	Versión base de planificación (antes de iniciar el desarrollo)

Contenido

1. Propósito y alcance
2. Roles y distribución del equipo
3. Ruta de trabajo
4. Identidad visual
5. Mockups de referencia
6. Estructura de archivos y carpetas
7. Contrato técnico compartido
8. Fases de construcción por rol
9. Flujo de trabajo en GitHub
10. Riesgos y cómo prevenirlos
11. Lista de verificación de la entrega
Anexo A. Datos de los integrantes

1. Propósito y alcance

Este documento fija, antes de escribir la primera línea de código, qué construye cada integrante, en qué orden y con qué reglas comunes. La práctica pide un sitio de dos páginas sobre una serie: una página principal con la presentación del equipo y una página de personajes que consume una API pública. Como el trabajo se reparte entre seis personas que tocan archivos distintos de un mismo proyecto, el riesgo real no es técnico sino de coordinación: nombres de clases que no coinciden, rutas rotas, estilos que se pisan. Por eso la guía incluye un contrato técnico compartido (sección 7) que los cuatro desarrolladores deben respetar.

La temática elegida es **Game of Thrones**, que es la serie que expone la API indicada en el enunciado. Toda la identidad visual (colores, tipografía, tono de los componentes) sale de esa temática y no de la plantilla institucional ni de una paleta genérica. Si el grupo decide trabajar con otra serie, basta cambiar la sección 4 y el endpoint de la sección 7; el resto del plan se mantiene.

La siguiente tabla traduce cada exigencia de la guía de práctica a una decisión concreta del proyecto y a la fase que la cumple. Sirve también como lista de control para la revisión final.

Requisito de la guía	Cómo se cumple en el proyecto	Fase
Sitio de 2 páginas (principal y personajes)	`index.html` y `personajes.html`, enlazadas entre sí desde la barra de navegación	1, 2
Tecnologías mínimas: HTML, CSS o Bootstrap y JavaScript; sin frameworks	Bootstrap 5 por CDN solo para rejilla y navbar; apariencia propia en `styles.css`; JavaScript puro (ES6)	3, 4
No usar base de datos	Los datos de los integrantes van escritos en el HTML; los personajes llegan por `fetch` a la API	1, 4
Página principal: nombre de la serie, logo, menú, «Miembros del equipo # 21» y tarjetas con imagen, nombre, carrera, nivel, correo y paralelo de todos los integrantes	Seis tarjetas en rejilla de 3 columnas (ver mockup 1)	1, 3
Página de personajes: mínimo 16, con imagen, nombre y al menos 3 datos más	Tarjetas generadas por JS con título, casa, nombre, apellido e ID (ver mockup 2)	2, 4
Personajes según el documento de asignación del docente	Pendiente de confirmar: el docente expone un documento con la asignación por grupo. Se carga en una constante del JS (sección 8, fase 4)	4
Paleta libre por grupo	Paleta temática definida en la sección 4	3
Informe PDF con carátula, objetivos, introducción, desarrollo, resultados, conclusiones y bibliografía	Estructura y calendario de capturas en la fase 6	6
Anexar el proyecto	ZIP del repositorio generado tras la revisión final	5, 6
Práctica grupal; plagio o copia = nota cero	Código propio, comentarios propios y API citada como fuente. Revisión de originalidad en la fase 5	Todas

Una observación sobre la tabla: el requisito del documento de asignación es el único que hoy no se puede cerrar, porque depende de un archivo que el docente muestra aparte. Conviene resolverlo antes de que la fase 4 empiece a programar el listado.

2. Roles y distribución del equipo

Cuatro integrantes se reparten los archivos principales, uno responde por la revisión y estandarización del código y otro por el informe. Cada rol funciona como una fase del proyecto, con su propia rama en GitHub y su propio entregable. Las fases no son estrictamente secuenciales: la 1 y la 2 arrancan juntas, la 3 y la 4 se solapan con ellas gracias al contrato técnico, y la 6 empieza desde el primer día recogiendo capturas.

Fase	Rol	Responsable	Archivos a su cargo	Entregable
1	Maquetación de la página principal	[Integrante 1]	`index.html`, `img/integrantes/`	Página principal completa con las 6 tarjetas
2	Maquetación de la página de personajes	[Integrante 2]	`personajes.html`	Estructura de la página con controles y contenedor del listado
3	Estilos y diseño visual	[Integrante 3]	`css/styles.css`, `img/logo/`	Hoja de estilos con la paleta, componentes y diseño responsive
4	Lógica y consumo de la API	[Integrante 4]	`js/main.js`, `js/personajes.js`	Listado dinámico con búsqueda, filtro y manejo de errores
5	Revisión y estandarización	[Integrante 5]	Todo el repositorio (solo revisa y fusiona), `README.md`	Código unificado, validado y paquete final del proyecto
6	Informe	[Integrante 6]	`docs/informe/`, `docs/capturas/`	Informe en PDF según la rúbrica

Los corchetes se reemplazan con los nombres reales apenas el grupo confirme el reparto. Un detalle práctico: el responsable de la fase 5 también hace de puerta de entrada a la rama principal, es decir, nadie fusiona a `main` sin su aprobación. Quien ya acostumbra a coordinar trabajos grupales encaja bien en ese rol, aunque la decisión es del grupo.

3. Ruta de trabajo

La guía de práctica asigna 9 horas de actividad, así que el calendario se plantea por hitos y no por fechas; cada grupo lo ajusta al plazo que muestre la plataforma. La lógica es sencilla: primero se acuerdan las reglas, luego se construye en paralelo apoyándose en el contrato técnico, después se integra y por último se revisa y se empaqueta. El informe corre en carril aparte durante todo el proceso.

Hito	Qué ocurre	Quién	Condición para pasar al siguiente
0. Arranque	Reunión corta: confirmar tema, repartir fases, recopilar los datos de los 6 integrantes (anexo A), dar acceso al repositorio y crear la estructura base de carpetas con archivos vacíos	Todos; la estructura base la sube la fase 5	Todos pueden clonar el repo y ven las mismas carpetas
1. Esqueletos	Se escribe el HTML de las dos páginas siguiendo el contrato técnico	Fases 1 y 2	Ambas páginas abren en el navegador con la estructura correcta (sin estilo todavía)

Hito	Qué ocurre	Quién	Condición para pasar al siguiente
2. Construcción paralela	Se redactan los estilos y la lógica mientras el HTML se termina de pulir	Fases 3 y 4	Las páginas se ven con la paleta y el listado de personajes carga desde la API
3. Integración	Se prueba todo junto, se corrigen desajustes de nombres de clases e IDs	Fases 1 a 4, coordinados por la 5	Sin errores en la consola y comportamiento correcto en tres anchos de pantalla
4. Revisión	Estandarización, validadores, pruebas de apertura local y comprobación de originalidad	Fase 5	Lista de verificación de la sección 8 (fase 5) completa
5. Cierre	Capturas finales, informe terminado, ZIP del proyecto, subida a la plataforma	Fases 5 y 6	PDF y proyecto anexo subidos antes del plazo

El hito 3 es el que suele fallar en prácticas de este tipo, porque es cuando aparecen las diferencias entre lo que cada quien entendió. Reservar un bloque de tiempo explícito para integrar evita que esas diferencias se descubran el último día.

4. Identidad visual

4.1 Concepto

La estética parte de tres referencias de la serie: la oscuridad del Trono de Hierro como fondo, el oro de la casa Lannister como color de énfasis y el rojo profundo de la casa Targaryen para acciones y estados activos. El azul hielo de los Stark se reserva para el texto secundario y los bordes. El resultado es un tema oscuro de contraste alto, con tipografía serif de aire medieval en los títulos y una serif legible en el cuerpo. No se utilizan el azul marino ni el naranja de la plantilla institucional.

4.2 Paleta de colores

Cada color tiene un nombre propio, un valor hexadecimal y una variable CSS. La fase 3 declara las variables una sola vez en `:root` y el resto del código las consume; nadie escribe hexadecimales sueltos en el HTML ni en otras reglas.

Color	HEX	Variable CSS	Uso
Negro Trono de Hierro	#0E0E10	`--negro-hierro`	Fondo general de la página
Grafito de Invernalia	#1C1D21	`--grafito`	Barra superior, tarjetas y pie de página
Oro Lannister	#C9A24B	`--oro`	Títulos de sección, etiquetas, líneas decorativas
Rojo Targaryen	#9B1C1F	`--rojo`	Enlace activo, botones, foco y hover (solo como relleno o borde)
Pergamino	#E9DFC7	`--pergamino`	Texto principal sobre fondos oscuros
Azul Hielo Stark	#8FA9B8	`--hielo`	Texto secundario, datos de las tarjetas, bordes en hover
Gris Acero	#5B6168	`--acero`	Bordes sutiles, marcadores de posición, texto deshabilitado

Una regla de contraste que conviene respetar: el pergamino, el oro y el azul hielo se leen bien sobre negro y grafito, pero el rojo Targaryen **no** debe usarse como color de texto sobre fondo oscuro porque su contraste queda muy por debajo de lo recomendable. El rojo vive como relleno de botones, subrayado del enlace activo y borde de foco.

```
:root {
  --negro-hierro: #0E0E10;
  --grafito:      #1C1D21;
  --oro:          #C9A24B;
  --rojo:         #9B1C1F;
  --pergamino:    #E9DFC7;
  --hielo:        #8FA9B8;
  --acero:        #5B6168;
}
```

4.3 Tipografía y componentes

Los títulos y el nombre de la serie usan **Cinzel** (Google Fonts), de trazo clásico y mayúsculas elegantes. El cuerpo de texto usa **Lora** o **Crimson Text**, ambas serif con buena legibilidad en pantalla. Siempre se declara una tipografía de respaldo (`Georgia, serif`) por si la fuente externa no carga en el computador desde donde se evalúe. Las tarjetas llevan fondo grafito, borde fino de acero, una línea superior o lateral dorada y esquinas casi rectas (radio de 4 px) para mantener el tono sobrio; al pasar el cursor, el borde cambia a azul hielo y la tarjeta se eleva unos pocos píxeles. Todos los elementos interactivos muestran un anillo de foco rojo o dorado visible con teclado.

5. Mockups de referencia

Los dos mockups siguientes son la referencia visual final de cada página, trazada sobre un ancho de escritorio de 1280 px. Reproducen la distribución que exige la guía de práctica (nombre de la serie, logo, menú «Principal | Personajes», título de sección, rejilla de tarjetas) y le aplican la paleta de la sección 4. Las fotos son marcadores de posición; los textos entre guiones se llenan con datos reales. Los círculos numerados remiten a las tablas que acompañan a cada imagen.

5.1 Página principal (index.html)

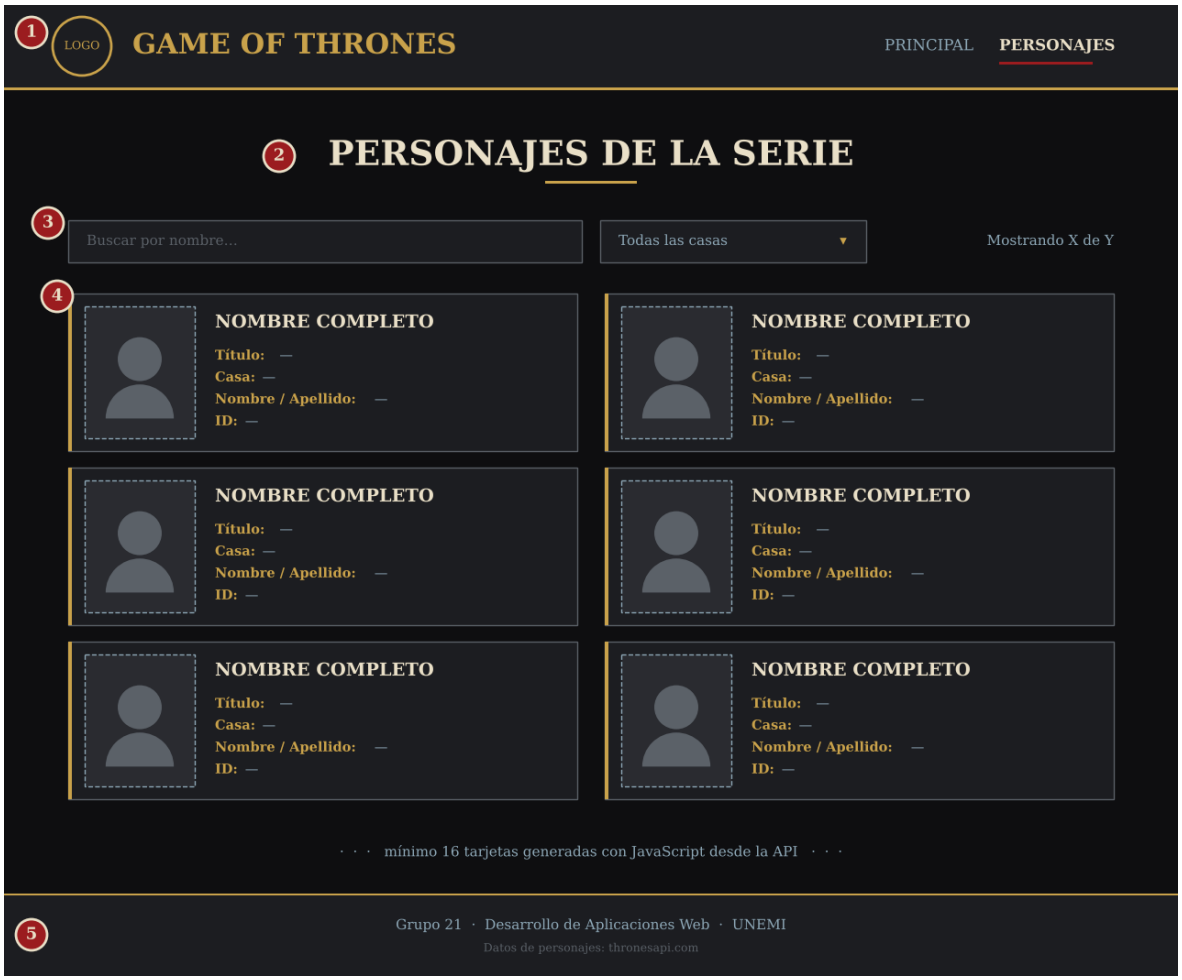


Mockup 1. Página principal con la presentación de los seis integrantes.

N.º	Sección	Contenido y distribución	Acciones y comportamiento
1	Encabezado	Logo de la serie (círculo con emblema propio) a la izquierda y nombre «Game of Thrones» en Cinzel dorado. Barra grafito con línea inferior dorada	El logo y el nombre llevan a `index.html`. Barra fija arriba al hacer scroll (opcional)
2	Navegación	Dos enlaces: Principal y Personajes, a la derecha del encabezado	El enlace de la página actual va en pergamino con subrayado rojo; el otro en azul hielo. Hover cambia el color. En móvil se colapsa en menú hamburguesa de Bootstrap
3	Título de sección	«Miembros del equipo # 21» centrado, con filete dorado y una línea secundaria con la materia	Solo texto
4	Tarjetas de integrantes	Rejilla de 3 columnas por 2 filas. Cada tarjeta: foto, nombre, carrera, nivel, correo y paralelo. Línea dorada superior	Hover con elevación y borde azul hielo. El correo es un enlace `mailto:`. En tablet pasa a 2 columnas y en móvil a 1
5	Pie de página	Texto «Grupo 21 · Desarrollo de Aplicaciones Web · UNEMI» y crédito de la API	El crédito enlaza a thronesapi.com en una pestaña nueva

Dentro de cada tarjeta, el orden de los datos es fijo (carrera, nivel, correo, paralelo) para que los seis integrantes queden alineados. Si el correo es largo, se permite el salto de línea con `overflow-wrap: anywhere` en lugar de recortar el texto.

5.2 Página de personajes (personajes.html)



Mockup 2. Página de personajes con controles de búsqueda y filtro.

N.º	Sección	Contenido y distribución	Acciones y comportamiento
1	Encabezado y navegación	Idéntico al de la página principal; ahora el enlace activo es Personajes	Mismo bloque HTML y mismas clases que en `index.html`
2	Título de sección	«Personajes de la serie» centrado con filete dorado	Solo texto
3	Controles	Campo de búsqueda por nombre, selector de casa y contador «Mostrando X de Y»	La búsqueda filtra mientras se escribe. El selector se llena con las casas presentes en los datos. El contador se actualiza con cada cambio
4	Listado de personajes	Rejilla de 2 columnas (1 en móvil) con tarjetas horizontales: imagen a la izquierda; a la derecha nombre completo, título, casa, nombre, apellido e ID	Tarjetas generadas por JS. Mínimo 16 visibles. Si una imagen falla, se muestra una imagen de respaldo. Mensajes de «Cargando…» y de error con botón Reintentar en el contenedor `#estado`
5	Pie de página	Igual que en la página principal	Mismo bloque HTML

Los controles de búsqueda y filtro no aparecen en el diseño base de la guía; se añaden porque aportan interacción real con JavaScript y ayudan a justificar el apartado de desarrollo del informe. Si el tiempo aprieta, se pueden recortar sin afectar los requisitos mínimos.

6. Estructura de archivos y carpetas

El repositorio se organiza con las dos páginas HTML en la raíz y las carpetas de recursos separadas por tipo. Se usan rutas relativas en todo el proyecto para que el sitio funcione al abrirlo con doble clic desde cualquier computador, que es justamente como lo va a verificar el docente. Por la misma razón los scripts se cargan como scripts clásicos con `defer` y **no** como módulos ES (`type="module"`), porque los módulos no funcionan al abrir un archivo con el protocolo `file://`.

```
DAW_GRUPO_21_S10-TPE_3/
├── index.html                 # Fase 1: página principal
├── personajes.html            # Fase 2: página de personajes
├── README.md                  # Fase 5: descripción, integrantes y cómo abrir el sitio
├── .gitignore                 # Fase 5: archivos que no se suben
├── css/
│   └── styles.css             # Fase 3: variables, base, componentes y responsive
├── js/
│   ├── main.js                # Fase 4: comportamiento común (año del pie, etc.)
│   └── personajes.js          # Fase 4: consumo de la API, filtros y renderizado
├── img/
│   ├── logo/                  # Fase 3: logo de la serie (emblema propio)
│   ├── integrantes/           # Fase 1: fotos de los 6 integrantes
│   └── placeholder-personaje.png  # Fase 3: imagen de respaldo si falla la API
└── docs/
    ├── informe/               # Fase 6: borradores y PDF final
    ├── capturas/              # Todos: capturas para el informe
    └── guia-planificacion.pdf # Este documento
```

Para evitar problemas de compatibilidad entre sistemas operativos, los nombres de archivo van en minúsculas, sin espacios, sin tildes ni ñ y con guiones, por ejemplo `integrante-01.jpg` o `f3-estilos-01.png`. Las fotos de los integrantes se redimensionan a unos 400 px de ancho y se mantienen por debajo de 200 KB; un sitio con seis fotos de 5 MB tarda en cargar y engorda el repositorio sin necesidad.

Ruta	Responsable	Regla principal
`index.html`	Fase 1	Solo HTML semántico; sin estilos en línea ni scripts embebidos
`personajes.html`	Fase 2	Mismo encabezado, navegación y pie que `index.html`, carácter por carácter
`css/styles.css`	Fase 3	Un único archivo, ordenado por secciones comentadas; todos los colores vía variables
`js/main.js`	Fase 4	Código común a ambas páginas; no depende de `personajes.js`
`js/personajes.js`	Fase 4	Solo se carga en `personajes.html`; sin variables globales sueltas
`img/`	Fases 1 y 3	Nombres en minúsculas y con guiones; imágenes optimizadas
`docs/`	Fase 6 (capturas: todos)	Nada de código fuente del sitio en esta carpeta

7. Contrato técnico compartido

Este apartado es lo que permite que cuatro personas trabajen a la vez sin esperarse. Define los identificadores, clases y funciones que cruzan fronteras entre archivos: lo que el HTML expone y lo que el CSS y el JS esperan encontrar. Cualquier cambio sobre estos nombres se avisa al grupo antes de hacerlo.

7.1 Identificadores y clases

Elemento	Selector	Lo define	Lo usan
Barra de navegación	`#navbar`, `.nav-link` (con `.active` en la página actual)	Fases 1 y 2	Fase 3
Contenedor de tarjetas de integrantes	`#integrantes` (fila de Bootstrap)	Fase 1	Fase 3
Tarjeta de integrante	`.card-integrante` con `__foto`, `__nombre`, `__dato`	Fase 1	Fase 3
Campo de búsqueda	`#buscador`	Fase 2	Fases 3 y 4
Selector de casa	`#filtro-casa`	Fase 2	Fases 3 y 4
Contador del listado	`#contador`	Fase 2	Fases 3 y 4
Mensajes de carga y error	`#estado`	Fase 2	Fases 3 y 4
Contenedor del listado	`#lista-personajes` (fila de Bootstrap)	Fase 2	Fases 3 y 4
Tarjeta de personaje (generada por JS)	`.card-personaje` con `__imagen`, `__cuerpo`, `__nombre`, `__dato`	Fase 4 (estructura), fase 3 (estilo)	Fase 3

La convención de nombres sigue la idea de BEM simplificado: bloque, doble guion bajo y elemento (`card-personaje__nombre`). Es más verboso que escribir `.nombre`, pero evita que los estilos de una tarjeta contaminen a la otra.

7.2 Estructura HTML de la tarjeta de personaje

La fase 4 genera esta estructura desde JavaScript y la fase 3 la estiliza. Los valores del ejemplo son ilustrativos.

```
<div class="col">
  <article class="card-personaje">
    <img class="card-personaje__imagen" src="URL_DE_LA_IMAGEN"
         alt="Retrato de NOMBRE COMPLETO" loading="lazy">
    <div class="card-personaje__cuerpo">
      <h3 class="card-personaje__nombre">Nombre completo</h3>
      <p class="card-personaje__dato"><span>Título:</span> ...</p>
      <p class="card-personaje__dato"><span>Casa:</span> ...</p>
      <p class="card-personaje__dato"><span>Nombre:</span> ...</p>
      <p class="card-personaje__dato"><span>Apellido:</span> ...</p>
      <p class="card-personaje__dato"><span>ID:</span> ...</p>
    </div>
  </article>
</div>
```

7.3 API y datos

El endpoint de la lista completa es `https://thronesapi.com/api/v2/Characters`. Según la documentación pública, cada objeto trae los campos `id`, `firstName`, `lastName`, `fullName`, `title`, `family`, `image` e `imageUrl`. Esto debe verificarse al inicio de la fase 4 abriendo la URL en el navegador, porque son los nombres exactos los que usará el código. También conviene revisar el campo `family`: en este tipo de APIs es frecuente que una misma casa aparezca escrita de varias formas, y eso ensucia el selector de casas si no se normaliza.

```
const URL_API = 'https://thronesapi.com/api/v2/Characters';

async function cargarPersonajes() {
  mostrarEstado('Cargando personajes...');
  try {
    const respuesta = await fetch(URL_API);
    if (!respuesta.ok) throw new Error('HTTP ' + respuesta.status);
    const datos = await respuesta.json();
    personajes = datos;            // lista completa en memoria
    llenarFiltroCasas(personajes);
    renderizar(personajes);
    mostrarEstado('');
  } catch (error) {
    mostrarEstado('No se pudo cargar la información. Intenta de nuevo.', true);
  }
}
```

Los datos de los integrantes de la página principal **no** pasan por JavaScript: se escriben directamente en el HTML. Así se cumple la restricción de no usar base de datos de la forma más simple posible y se evita que la página principal dependa de la conexión.

8. Fases de construcción por rol

Cada fase se describe con el mismo molde: objetivo, tareas con su criterio de aceptación, rama de trabajo y un ejemplo del mensaje de commit. El criterio de aceptación es lo que la fase 5 comprueba antes de dar por buena la entrega.

Fase 1. Página principal (index.html)

El objetivo es dejar lista la presentación del equipo con la estructura exacta del contrato técnico. Esta fase también recopila las fotos y datos de los seis integrantes, porque sin ellos no se puede cerrar la página. Rama: `feat/index`.

Tarea	Detalle	Criterio de aceptación
Cabecera del documento	`lang="es"`, `charset`, `viewport`, título, enlaces a Bootstrap (CDN con versión fija), Google Fonts y `css/styles.css`	La página abre sin avisos y los recursos se cargan por rutas relativas
Encabezado y navegación	Logo, nombre de la serie y enlaces Principal y Personajes con `.active` en Principal	Los enlaces llevan a las dos páginas
Título de sección	«Miembros del equipo # 21»	Texto exacto según el diseño
Tarjetas de integrantes	Seis tarjetas con foto, nombre, carrera, nivel, correo y paralelo	Los seis integrantes aparecen, sin datos faltantes; `alt` descriptivo en cada foto
Pie de página	Texto del grupo y crédito de la API	Idéntico al de `personajes.html`

Tarea	Detalle	Criterio de aceptación
Carga de scripts	`<script src="js/main.js" defer>` al final del `body`	Sin errores en consola

Commit de ejemplo: `feat(index): agrega estructura y tarjetas de integrantes`.

Fase 2. Página de personajes (personajes.html)

Aquí se construye el armazón de la página que luego llena el JavaScript. Es importante no escribir personajes a mano: el HTML solo define los contenedores y controles. Rama: `feat/personajes`.

Tarea	Detalle	Criterio de aceptación
Cabecera, navegación y pie	Copiar los bloques de `index.html` cambiando solo el enlace activo	Al comparar con `index.html`, solo difiere `.active` y el título de la pestaña
Título de sección	«Personajes de la serie»	Texto exacto según el diseño
Barra de controles	`#buscador`, `#filtro-casa` y `#contador` con `label` asociados	Cada control tiene su etiqueta, aunque sea oculta visualmente
Zona de estado	`#estado` con `aria-live="polite"`	Existe y está vacío por defecto
Contenedor del listado	`#lista-personajes` como `div.row`	Vacío; sin tarjetas escritas a mano
Carga de scripts	`main.js` y `personajes.js`, ambos con `defer`	Orden correcto y sin errores de ruta

Commit de ejemplo: `feat(personajes): agrega controles y contenedor del listado`.

Fase 3. Estilos y diseño visual (styles.css)

La fase 3 convierte las maquetas en el aspecto de los mockups. Trabaja contra el contrato técnico, así que puede empezar en cuanto se suba el esqueleto del HTML, incluso con tarjetas de prueba. También prepara el logo propio y la imagen de respaldo para personajes. Rama: `feat/css`.

Tarea	Detalle	Criterio de aceptación
Variables y base	Bloque `:root` con la paleta, fuente del cuerpo, fondo negro, color de texto pergamino	No aparecen hexadecimales fuera de `:root`
Tipografía	Cinzel en títulos y Lora o Crimson Text en el cuerpo, con tipografía de respaldo	Se ve bien incluso sin conexión a Google Fonts
Encabezado y navegación	Barra grafito con línea dorada, estados normal, hover y activo	Coincide con los mockups
Tarjeta de integrante	Fondo, borde, línea dorada, foto centrada, datos alineados, hover	Las seis tarjetas tienen la misma altura visual
Tarjeta de personaje	Disposición horizontal, imagen con proporción fija (`object-fit: cover`)	Imágenes sin deformar y textos legibles
Controles y estados	Estilo de campo de búsqueda, selector, mensaje de carga y de error	Foco visible con teclado
Responsive	Puntos de quiebre en 576, 768 y 992 px	Sin desplazamiento horizontal en 375, 768 y 1280 px de ancho

Tarea	Detalle	Criterio de aceptación
Logo y recursos	Emblema propio en `img/logo/` e imagen de respaldo	El logo se ve nítido en el encabezado

Un aviso sobre el logo: los logotipos oficiales de la serie son material protegido. Lo más seguro para un trabajo académico es crear un emblema sencillo propio (por ejemplo, una corona o un escudo con las iniciales) y mencionarlo así en el informe. Commit de ejemplo: `feat(css): define paleta y estilos de tarjetas`.

Fase 4. Lógica y consumo de la API (js)

Es la fase con más carga técnica. Se divide en dos archivos: `main.js`, con lo común a ambas páginas, y `personajes.js`, con todo lo relativo a la API. Antes de programar el listado hay que resolver el pendiente del documento de asignación: si el docente fija qué personajes corresponden al Grupo 21, esa lista se guarda en una constante `PERSONAJES_ASIGNADOS` y la página muestra solo esos, con un mínimo de 16. Rama: `feat/js`.

Tarea	Detalle	Criterio de aceptación
Verificar la API	Abrir el endpoint, confirmar nombres de campos y cantidad de registros	Los campos usados en el código existen en la respuesta
Consumo con `fetch`	Función asíncrona con `try/catch` y comprobación de `respuesta.ok`	Un fallo de red muestra el mensaje de error en `#estado`
Renderizado	Construir las tarjetas con `createElement` y `textContent` (no `innerHTML` con datos de la API)	Al menos 16 tarjetas con imagen, nombre y tres datos más
Búsqueda por nombre	Filtrado en memoria al escribir en `#buscador`, ignorando mayúsculas y tildes	El contador refleja el resultado
Filtro por casa	Selector llenado dinámicamente, con normalización de nombres de casa	No aparecen casas duplicadas por diferencias de escritura
Imágenes	`onerror` que sustituye por la imagen de respaldo	Ninguna tarjeta queda con imagen rota
Estados y reintento	Mensaje de carga, de error y botón Reintentar	Se puede simular el error desconectando la red
`main.js`	Año actual en el pie y cualquier comportamiento común	No depende de `personajes.js`

Restricciones a respetar: JavaScript puro, sin librerías de terceros, sin `type="module"`, sin guardar datos en bases de datos ni en almacenamiento persistente. Comentarios en español y escritos por quien programa; el código copiado tal cual de internet es justamente lo que dispara la alerta de plagio. Commit de ejemplo: `feat(js): consume la API y renderiza personajes`.

Fase 5. Revisión y estandarización

El quinto integrante tiene dos funciones: acompañar el proceso, es decir, abrir las solicitudes de fusión (pull requests) de cada rama, revisarlas y fusionarlas a `main`, y hacer la pasada final de calidad. Además crea la estructura base en el hito 0 y redacta el `README.md`. Rama: `review/estandarizacion`.

Aspecto	Qué se verifica	Cómo
Estructura	Carpetas y nombres según la sección 6	Comparación con el árbol del documento
HTML	Documento válido, semántico, con `alt` en las imágenes y sin estilos en línea	Validador del W3C (validator.w3.org)
CSS	Variables usadas, sin `!important` injustificado, sin reglas duplicadas	Validador CSS del W3C y lectura del archivo
JavaScript	Sin errores en consola, sin variables globales innecesarias, funciones con nombres claros	Consola de las herramientas del navegador
Formato	Indentación de 2 espacios, comillas dobles en HTML, nombres coherentes, comentarios en español	Revisión manual; opcional `.editorconfig`
Coherencia	Encabezado y pie idénticos en ambas páginas; enlaces entre páginas funcionando	Comparación lado a lado
Rutas	Todas relativas; ninguna apunta a `C:\` ni a carpetas personales	Búsqueda de `C:` y `file:///` en el proyecto
Apertura local	El sitio funciona con doble clic en `index.html` y también con un servidor local	Prueba en Chrome y en un segundo navegador
Responsive	Se ve bien en 375, 768 y 1280 px de ancho	Modo dispositivo de las herramientas del navegador
Requisitos de la guía	Seis integrantes con todos sus datos, mínimo 16 personajes con 3 datos más, sin frameworks prohibidos, sin base de datos	Lista de la sección 1
Originalidad	Código y textos propios, API citada como fuente, sin fragmentos copiados sin entender	Preguntar a cada autor qué hace su código; revisar comentarios
Paquete final	ZIP del proyecto sin carpetas innecesarias, probado en una carpeta distinta	Descomprimir y abrir desde otra ubicación

Esta fase cuida que la revisión no se convierta en reescritura: se señalan los problemas y se devuelven a su autor con un comentario en la solicitud de fusión, salvo errores menores de formato que sí puede corregir directamente. Commit de ejemplo: `chore(review): unifica formato y corrige rutas`.

Fase 6. Informe

El informe corre en paralelo desde el hito 0. La condición para que no se acumule al final es que cada responsable de fase le entregue, apenas termine su parte, capturas del código y un párrafo explicando qué hizo y por qué. Las capturas se guardan en `docs/capturas/` con nombre `fX-descripcion-NN.png` (por ejemplo `f4-fetch-01.png`). Si el docente o la plataforma exigen una carátula institucional para el informe, se usa esa; para esta guía interna no hace falta. Rama: `docs/informe`.

La rúbrica de la práctica suma 15 puntos, a razón de 3 por criterio. La tabla relaciona cada criterio con lo que el informe debe contener.

Criterio (3 pts c/u)	Qué debe mostrar el informe	Fuente del contenido
Desarrollo	Explicación de cómo se construyó cada archivo, con capturas del código y descripción de las acciones realizadas	Capturas y párrafos de las fases 1 a 4

Criterio (3 pts c/u)	Qué debe mostrar el informe	Fuente del contenido
Redacción y ortografía	Texto sin errores, con párrafos claros y terminología técnica consistente	Revisión final de la fase 6, con lectura cruzada de otro integrante
Resultados	Capturas del resultado final en el navegador: ambas páginas, el listado de personajes, la vista móvil, el manejo de errores	Capturas del hito 3 y 4
Conclusiones y recomendaciones	Conclusiones coherentes con lo desarrollado: qué se logró con HTML, CSS y JS, qué se aprendió del consumo de la API, qué se mejoraría	Reunión corta del grupo al cierre
Anexos	Proyecto completo del sitio anexado, tal como pide la guía	ZIP de la fase 5

Estructura del informe, en el orden que pide la guía: carátula, objetivos (los dos de la guía, redactados con palabras propias), introducción, desarrollo, resultados, conclusiones y recomendaciones, y bibliografía en formato APA 7. Como mínimo la bibliografía incluye la referencia que figura en la guía (Guzdial, M. J., y Ericson, B. (2013). Introducción a la programación con Python: Un enfoque multimedia. Pearson), la documentación de la API (thronesapi.com), la de Bootstrap y la de MDN Web Docs. Cada fuente se cita solo si de verdad se usó, y los datos exactos de las referencias web (fechas de consulta, enlaces) se confirman al momento de redactar.

9. Flujo de trabajo en GitHub

El repositorio pertenece a la cuenta MateoACantu, así que el propietario debe invitar a los otros cinco integrantes como colaboradores (en GitHub: Settings, Collaborators) antes del hito 0; sin eso nadie más podrá subir cambios. Cada fase trabaja en su rama y nunca sube directamente a `main`. Cuando una parte está lista, se abre una solicitud de fusión hacia `main` y la fase 5 la revisa.

Rama	Fase	Contenido
`main`	Fase 5 fusiona	Versión estable e integrada; siempre abre sin errores
`feat/index`	1	`index.html` y fotos de integrantes
`feat/personajes`	2	`personajes.html`
`feat/css`	3	`css/styles.css`, logo y recursos visuales
`feat/js`	4	`js/main.js` y `js/personajes.js`
`review/estandarizacion`	5	Correcciones de formato, `README.md`, `.gitignore`
`docs/informe`	6	Borradores del informe y capturas

Los comandos básicos de cada integrante son los siguientes. Lo más importante es traer los cambios de `main` antes de subir los propios, para evitar conflictos grandes al fusionar.

```
# 1. Clonar el repositorio (solo la primera vez)
git clone https://github.com/MateoACantu/DAW_GRUPO_21_S10-TPE_3.git
cd DAW_GRUPO_21_S10-TPE_3

# 2. Crear y cambiar a tu rama de fase
git checkout -b feat/index
```

```
# 3. Trabajar, y cada cierto avance guardar y subir
git add index.html img/integrantes/
git commit -m "feat(index): agrega tarjetas de integrantes"
git pull origin main            # traer lo último de main antes de subir
git push -u origin feat/index

# 4. En GitHub: abrir Pull Request hacia main y avisar a la fase 5
```

Para los mensajes de commit se usa el formato `tipo(ámbito): descripción en minúsculas`, con los tipos `feat`, `fix`, `style`, `docs` y `chore`. Como `index.html`, `personajes.html` y `styles.css` son los archivos donde es más probable que dos personas toquen las mismas líneas, el criterio es simple: cada archivo tiene un solo dueño; si alguien necesita un cambio en el archivo de otro, lo pide por el grupo o lo comenta en la solicitud de fusión.

10. Riesgos y cómo prevenirlos

Riesgo	Consecuencia	Prevención
No hay internet en el computador del docente	La API, Bootstrap y las fuentes no cargan	Mensaje de error claro con reintento; fuentes de respaldo; avisar en el informe que el sitio requiere conexión
Rutas absolutas o `type="module"`	El sitio no abre con doble clic	Solo rutas relativas y scripts clásicos; prueba de apertura local en la fase 5
Nombres de clases o IDs que no coinciden	Estilos o JS que no se aplican	Respetar el contrato de la sección 7; integración en el hito 3
Conflictos de fusión en archivos compartidos	Pérdida de trabajo	Un dueño por archivo; traer `main` antes de subir
Personajes distintos a los asignados al grupo	Pérdida de puntos en el desarrollo	Confirmar el documento de asignación antes del hito 2
Imágenes pesadas o rotas	Carga lenta y tarjetas deformes	Optimizar fotos; imagen de respaldo en el JS
Informe dejado para el final	Capturas faltantes y redacción apurada	Entrega de capturas y párrafos por fase al terminar cada una
Sospecha de copia o plagio	Nota automática de cero	Código y textos propios, citas de fuentes, cada autor sabe explicar su código

11. Lista de verificación de la entrega

Antes de subir nada a la plataforma, el grupo recorre esta lista con el proyecto abierto en un computador distinto al de quien lo programó. Si algún punto falla, se devuelve a la fase correspondiente.

Verificación	Fase responsable	Listo
Las dos páginas abren con doble clic y se navega entre ellas	1, 2, 5	☐
La página principal muestra a los 6 integrantes con imagen, nombre, carrera, nivel, correo y paralelo	1	☐
El número de grupo «21» aparece en el título de la página principal	1	☐

Verificación	Fase responsable	Listo
La página de personajes muestra al menos 16 personajes con imagen, nombre y 3 datos más	4	☐
Los personajes corresponden a la asignación del docente	4	☐
La paleta temática se aplica en las dos páginas y se ve bien en móvil, tableta y escritorio	3	☐
Sin errores en la consola; sin frameworks prohibidos ni base de datos	4, 5	☐
Informe PDF con carátula, objetivos, introducción, desarrollo, resultados, conclusiones y bibliografía	6	☐
Proyecto anexado en un ZIP que abre desde otra carpeta	5	☐
Informe y proyecto subidos a la plataforma	Todos	☐

Anexo A. Datos de los integrantes (para recopilar en el hito 0)

Esta tabla se completa una sola vez y la fase 1 la usa para armar las tarjetas. La carrera, el nivel y el paralelo se escriben tal como figuran en la matrícula para que coincidan entre los seis.

Integrante	Fase	Carrera	Nivel	Correo institucional	Paralelo	Foto
Integrante 1	1 · index					integrante-01.jpg
Integrante 2	2 · personajes					integrante-02.jpg
Integrante 3	3 · css					integrante-03.jpg
Integrante 4	4 · js					integrante-04.jpg
Integrante 5	5 · revisión					integrante-05.jpg
Integrante 6	6 · informe					integrante-06.jpg

Una vez completada, se reemplaza «Integrante N» por el nombre completo de cada persona. En la guía del docente, la carrera corresponde a Tecnologías de la Información en modalidad en línea; se confirma con cada integrante antes de escribirla en las tarjetas.